NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

**BACHELOR'S THESIS**

**Research Project on the Topic:**

# Determining the Optimal Transformer Architecture for Recommender Systems in the Industrial Domain

**Prepared by:**

Student of the group БПАД211, Year 4
Spirina Mayya Alexandrovna

**Thesis Supervisor:**

Senior Developer
Tikhonovich Daria Mikhailovna

Moscow 2025

# Contents

**Annotation** **2**

**1 Introduction** **3**

1.1 Description of the Subject Area . . . . . 3

1.2 Problem Statement . . . . . 4

1.3 Research Aim and Results . . . . . 5

**2 Literature Review** **7**

2.1 Architecture Choice . . . . . 7

2.2 Existing Studies . . . . . 8

**3 Model Choice** **10**

3.1 Model Implementation . . . . . 10

3.2 Model Modifications . . . . . 12

3.3 Conclusions and Results . . . . . 15

**4 Performance Measurement** **16**

4.1 Datasets . . . . . 16

4.2 Metrics . . . . . 17

4.3 Conclusions and Results . . . . . 18

**5 Experiments** **19**

5.1 Experiment 1: Model Quality . . . . . 19

5.2 Experiment 2: Base Model . . . . . 20

5.3 Experiment 3: Benchmark of Modifications . . . . . 22

5.4 Experiment 4: Effect of Item Features . . . . . 28

5.5 Experiment 5: Effect of DCN network Item Features . . . . . 29

5.6 Conclusions and Results . . . . . 31

**6 Conclusion** **32**

**A Appendix** **33**

**References** **41**

# Annotation

Transformer-based models are widely employed in recommender systems for their pattern identification and scaling capabilities. However, due to architecture complexity it is challenging to identify component combinations that have the greatest impact on model performance. This study addresses this issue by developing a strong modular baseline model, upon which various modifications are compared and contribution of individual architecture parts is analysed. For model benchmarking time-based validation is applied with performance measured using both accuracy and beyond-accuracy metrics, making research suitable for industrial setting.

# Keywords

Recommender systems, transformer architecture, model benchmarking, time-based validation, beyond-accuracy metrics, popularity bias.

# 1 Introduction

## 1.1 Description of the Subject Area

With the rapid growth of online platforms, higher purchasing power and increase of available goods and services, the need for recommendation systems became clear. For the users of video services, online reading or e-commerce platforms may be hard to distinguish relevant items from existing options due to the large variety of products offered. To enhance both customer satisfaction and profitability of the services it is necessary to incorporate a recommendation system, which will offer personalized content.

Recommendation system is a model that takes previous user interactions and suggests the most relevant options for the client. There are 3 general types of recommendation systems: collaborative filtering, content filtering and hybrid recommender systems[13] distinguished by the data they use. This research focuses on collaborative filtering models that generate recommendations based solely on user-item interactions, as this allows to concentrate on architecture without results being affected by item/user feature choice. Conventional collaborative filtering models, such as Alternating Least Squares (ALS) or Singular Value Decomposition (SVD), provide strong baseline results, but lack expressiveness and flexibility required in large-scale settings. Thus, neural network architectures are commonly chosen as the main model.

Furthermore, since there is a natural ordering in which users interact with items, sequential task is introduced by many authors. For solving such a task transformer-based architectures are widely applied due to their ability to capture long-term dependencies. Transformer architecture consists of multiple components, including transformer layers, loss functions, training objectives, negative sampling strategies, etc. To choose an efficient model for candidate generation or ranking from a large variety of combinations extensive benchmarks using robust validation scheme, like the one applied in this work, are required.

## 1.2 Problem Statement

Transformer-based models provide a wide variety of architectural choices, each of them being potentially crucial for recommender model performance. However, the number of modifications is too large, making manual search infeasible. That is why researchers require guidance on the impact of individual components and their combinations, supported by reliable comparisons. Nevertheless, existing public benchmarks have a number of issues that limit insights that could be obtained:

- Incorrect implementations or lack of open-source implementations.
- Insufficient model tuning and provision of cherry-picked results to favor certain models.
- Incomplete comparison of model modifications.
- Usage of leave-one-out validation that leads to data leakage.
- Focus on ranking metrics (NDCG@K, MAP@K), not taking into account beyond-accuracy metrics that are crucial for estimating popularity bias and defining which model will succeed in A/B testing.

This research offers an extensive comparison of transformer model modifications based on robust validation scheme, which aims to solve above-mentioned issues.

For formal problem definition some notations will be introduced:

$$\mathcal{U} = \{u_1, u_2, \dots, u_N\} - \text{the set of users} \tag{1}$$

$$\mathcal{I} = \{i_1, i_2, \dots, i_M\} - \text{the set of items} \tag{2}$$

$$\mathcal{S}_u = [i_1^u, i_2^u, \dots, i_{T_u}^u] - \text{user sequence for user } u \in \mathcal{U}, \text{where } i_t^u \in \mathcal{I}. \tag{3}$$

$$\mathcal{S} = \begin{bmatrix} 0 & i_1^1 & i_2^1 & \dots & i_{T_1}^1 \\ 0 & \dots & \dots & \dots & \\ 0 & i_1^N & i_2^N & \dots & i_{T_N}^N \end{bmatrix} - \text{vector of padded user sequences.} \tag{4}$$

$$f_\theta : S \to \Re^{|U|\times|I|} - \text{model } f \text{ with parameter set of modifications } \theta \tag{5}$$

Model $f_\theta$ takes padded user sequences and maps them into scores for all items (logits). Based on logits, catalog items are sorted and recommendation list is formed. By comparing recommendation list to ground-truth data $\mathcal{G}$ metrics are computed. Let $\mathcal{M}$ be the set of accuracy and beyond-accuracy metrics: $\mathcal{M} = \{\text{Recall@10}, \text{ARP@10}\}$. Then, model performance can be measured by $\mathbf{m}_\theta = [m_1(f_\theta(\mathcal{S}), \mathcal{G}), m_2(f_\theta(\mathcal{S}), \mathcal{G})] \in \mathbb{R}^2, m_i \in \mathcal{M}$.

The task is to analyse parameters $\theta^* \in \Theta$ that achieve Pareto-optimal performance, i.e., there is no $\theta \in \Theta$:

$$m_{\theta,1} > m_{\theta^*,1}, m_{\theta,2} \leq m_{\theta^*,2} \text{ or } m_{\theta,1} \geq m_{\theta^*,1}, m_{\theta,2} < m_{\theta^*,2} \tag{6}$$

## 1.3 Research Aim and Results

### 1.3.1 Project Aim and Goals

Project aim is to benchmark impact of transformer model architecture components on recommendation system performance in a production setting. To achieve this goal, the following tasks were formulated:

- Review existing articles and transformer-based benchmarks. Analyse common comparison approaches.
- Implement robust open-source model with modular structure.
- Create a list of model modifications for comparison, implement them.
- Design and implement validation pipeline: choose validation scheme and metrics.
- Conduct experiments, analyse results, develop insights into optimal model architecture.
- Study the extent to which item features increase offline metrics, investigate effect of DCN network on beyond-accuracy metrics.

### 1.3.2 Results

As a result, modular model implementation that allows the flexibility to combine transformer architecture components was developed and released in open-source. Its quality was shown to be comparable to the solutions provided in existing frameworks and superior for some options. Based on this approach, comparison of modifications that were never measured against each other in one study was created. Another novel factor is that an extensive set of metrics, involving beyond-accuracy ones, is provided, allowing deeper investigation of the topic. Additionally to modification comparison, it was shown that item features are crucial for increasing beyond-accuracy metrics, this statement was supported with successful A/B test conducted by one of MTS services. Effect of item features was further studied by incorporation of DCN network into embedding creation procedure - method not benchmarked in the articles on open datasets. All of this gives guidance to researchers on how to choose optimal models and what architectures should be investigated more closely, which is important for the case when resources are limited and model has a large variability. Furthermore, both industrial developers and researchers could benefit from open-source implementation that can be easily modified for required purposes.

Open-source implementation can be found in RecTools[1] library, results of this research are located in the repository[2].

### 1.3.3 Research Structure

Section 2 describes relevant model architecture choices, discusses validation schemes and positions this study against other works that involve benchmarks. Next part concentrates on developing base model structure and designing components to be compared. After that, in section 4, datasets and metrics are defined that are used in the experiments (section 5) to measure model performance and define optimal transformer architecture.

[1] https://github.com/MobileTeleSystems/RecTools
[2] https://github.com/spirinamayya/TransformerModels/tree/main

# 2 Literature Review

## 2.1 Architecture Choice

Transformer-based models have been described by many authors in various forms. First vanilla approaches of using transformers to solve sequential task in recommender systems, such as SASRec [6] and BERT4Rec [23], were developed in 2018-2019. These models are based on a simple transformer with different training targets: shifted sequence prediction for SASRec and masked language modeling task for BERT4Rec. Since then, more advanced approaches have appeared that introduce improvements to transformer architecture. For instance, TIGER [19] applies generative retrieval with semantic ids and FuXI [28] modifies the self-attention mechanism to model temporal and positional features utilizing multistage FFN. By investigating changes introduced by the authors, main modifications were identified and grouped into several categories:

- **Training objective**. Models predict different objects: SASRec [6] applies autoregressive shifted sequence prediction, BERT4Rec [23] - masked language modeling, and PinnerFormer additionally discusses all action, next action and dense all action targets [12]. First 2 objectives are widely known and implemented, but dense all action requires further study.

- **Transformer layers**. This group involves large number of modifications. Recent examples are: ALBERT [7] layers that reduce the number of parameters for BERT4Rec or HSTU [30] that applies normalized attention.

- **Losses**. Cross-entropy loss on full catalogue of items (softmax loss) [23] is the most precise loss, as it is not biased. However, its computational infeasibility requires researchers to switch to other options. That is why researchers turn to losses with negative sampling: binary cross-entropy (BCE) [6], generalized binary cross-entropy (gBCE) [17] that accounts for the inherent popularity bias in losses with negative sampling, sampled-softmax [29].

- **Negative sampling strategy**. An extensive research on negative sampling [8] divides strategies into several groups: static, dynamic, adversarial generation, importance re-weighting and knowledge-enhanced. For sequential task most common are uniform sampling, in-batch sampling, in-batch sampling with log-Q correction needed to deal with popular items being sampled much more frequently, mixed negative sampling [27].

- **Similarity function**. It concerns how recommendations are calculated from logits. Choices involve dot-product and MLP [20], cosine similarity [10], mixture of logits [29]. This group lacks reproducible benchmarks.

This study investigates how model modifications influence performance.

## 2.2 Existing Studies

**Benchmarks of architecture modifications:**

With regard to similar studies from architecture point of view, they can be divided into several groups:

1 Articles that introduce new model and compare it with previous results. Such studies do not focus on individual components and take results without checking quality claimed by other authors [30], [31].

2 Benchmarks that study individual modifications more closely, for instance, "Revisiting Neural Retrieval on Accelerators" [29] investigates similarity functions, "PinnerFormer: Sequence Modeling for User Representation at Pinterest" [12] - training objectives. However, unlike this study, comparisons were limited to one component and no attempts to show combined effect were made.

This research provides a more extensive benchmark with modifications not previously compared against each other.

**Benchmarks from validation point of view:**

1 Unreproducible benchmarks. Recent studies outline the problem of unreproducible and unfair benchmarks in recommender systems [32]. Unreproducibility problem is solved in this study, as source code of implementations and validation is public, allowing researchers to repeat experiments.

2 Benchmarks that use leave-one-out validation. Leave-one-out validation scheme was shown to lead to data leakage. A more appropriate option is applying time-based splits [5], which this work will stick to, following [18].

3 Benchmarks using only accuracy metrics. Some benchmarks are not suitable for industrial purposes due to the metric choice. Most articles claim true positive metrics, for instance [2], which tend to be influenced by popularity bias [1]. At the same time popularity bias mitigation improves results of A/B testing [14]. Hence, this study evaluates models both with accuracy and beyond-accuracy metrics. For that, pareto-optimal models with respect to several metrics are investigated like it was done for iALS model in an article about curse of dimensionality [11].

4 Solid benchmarks - reproducible benchmarks which use time-based validation and state beyond-accuracy metrics. Such benchmarks, unlike this study, are not broad enough from architecture point of view [18], do not consider transformer architectures [21] or do not analyse beyond-accuracy metrics in greater details.

# 3 Model Choice

## 3.1 Model Implementation

First task was to develop a baseline model, because existing open-source implementations were not suitable for this research. They failed to satisfy a requirement for model to be modular so that modifications could be tested in a convenient and fair way. As a base model structure, standard transformer architecture used for solving sequential recommendation task was chosen. Figure 3.1 outlines its main parts.

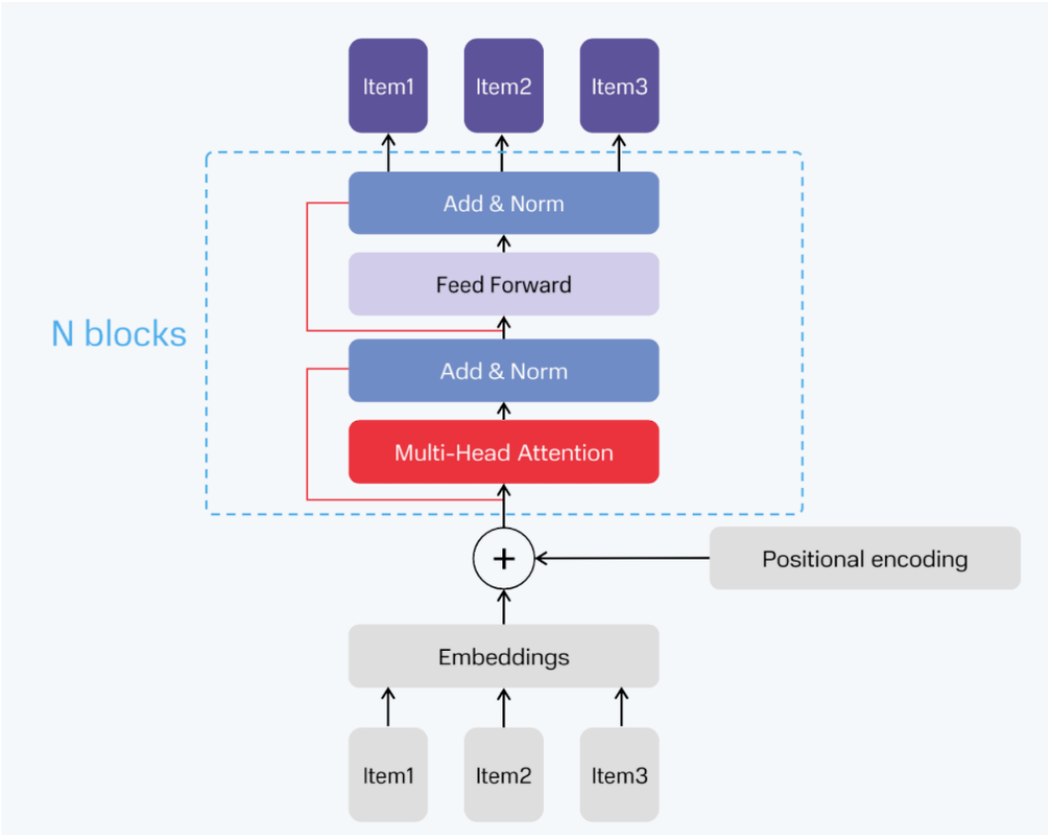


Figure 3.1: Transformer model in recommender systems

Initially user sequences $S_u = [i_1^u, i_2^u, \ldots, i_{T_u}^u]$ are obtained from user-item interactions by sorting them in chronological order. Each item id from user sequence is transformed into a vector of numbers (embeddings) and positional encoding embedding is added to take into account order of interactions, which is not preserved by default in transformer models. After that user sequence embeddings are passed into transformer blocks that may have varying architecture. What they have in common is multi-head attention [24] that consists of several attention layers .

In more details, the following class structure was designed and implemented 3.2, application example can be found in transformers_tutorial.ipynb file. All purple blocks can be replaced to modify components of the model without affecting other parts.

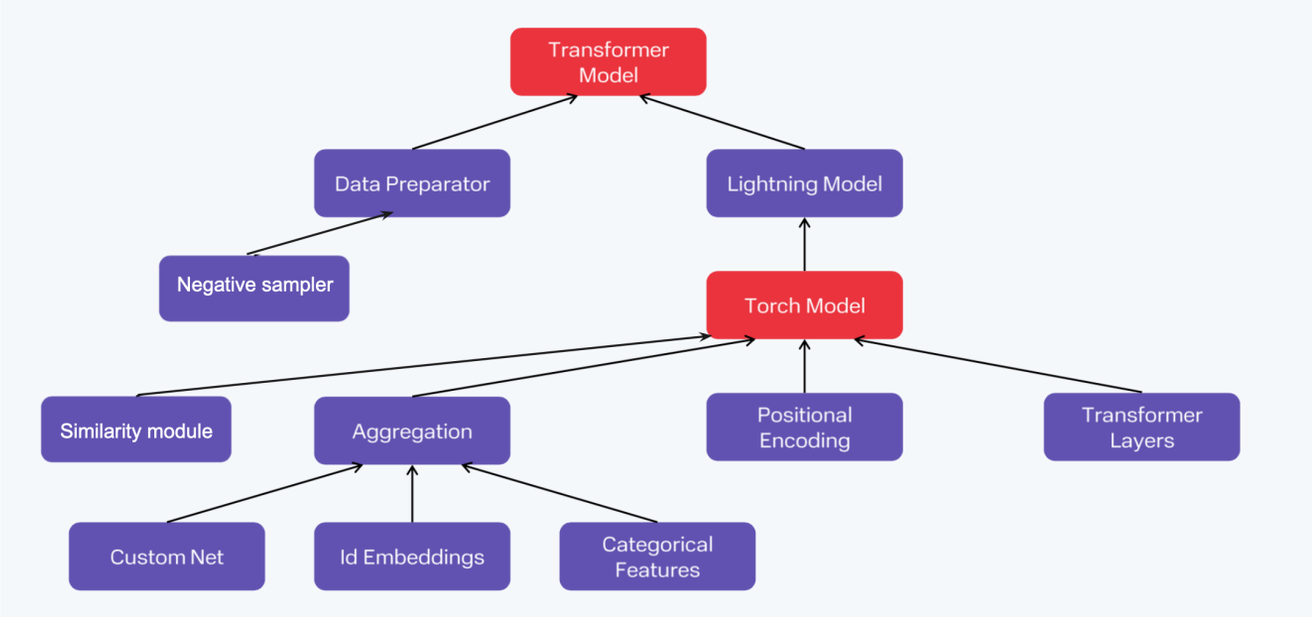


Figure 3.2: Base model blocks

Architecture was divided into 2 main blocks:

- **Data preparator.** Data preparator constructs user sequences from interactions, filters dataset leaving users with required history length and generates negatives for losses with sampling.

- **Model.** For model implementation PyTorch Lightning framework was used, as it offers stable trainer options (checkpoints, device transfer, optimizations).

  - Ligning module. Lightning module defines logic of torch model including loss function.

  - Item encoder networks encode each item id into embedding. Implementation allows passing item features, however, it is not used in benchmarks. Their effect is defined in a separate section.

  - Positional encoding. In this work it is fixed, inverse learnable position encoding is applied. This means that the most recent interaction receives id 1, second to the first 2, etc.

– Transformer layers in this study consist of multi-head attention from PyTorch framework with causal and/or padding mask, feed forward network with ReLU/GeLU activation function, normalization layers and skip connections.

– Similarity module defines how logits are computed from encoded user session (received from transformer) and candidate item embeddings.

## 3.2 Model Modifications

Based on literature review 5 model components were selected 3.1 :

Table 3.1: Model modifications

| Component | Option 1 | Option 2 | Option 3 | Option 4 |
|---|---|---|---|---|
| **Training objective** | Shifted sequence | MLM | All action | Dense all action |
| **Transformer layers** | SASRec | BERT4Rec | - | - |
| **Loss functions** | Softmax | Sampled softmax | BCE | gBCE |
| **Negative sampling** | Uniform | In-batch | In-batch + log-Q correction | Mixed |
| **Similarity function** | Dot product | Cosine | - | - |

1 Training objective.

1.1. *Shifted sequence.* Having user sequence $S_u = [i_1^u, i_2^u, \ldots, i_{T_u}^u]$, model predicts next item for each subsequence.

$$S_u^{\text{train}} = (i_1^u), \quad S_u^{\text{target}} = (i_2^u) \tag{7}$$

$$S_u^{\text{train}} = (i_1^u, i_2^u), \quad S_u^{\text{target}} = (i_3^u) \tag{8}$$

$$S_u^{\text{train}} = (i_1^u, i_2^u, ..., i_{T_u-1}^u), \quad S_u^{\text{target}} = (i_{T_u}^u) \tag{9}$$

1.2. *Masked language modeling (MLM).* For this task some items are randomly replaced by MASK token, following the same logic as in BERT models. Aim is to predict these masked tokens based on surrounding context.

$$S_u^{\text{train}} = (i_1^u, MASK, ..., i_{T_u-1}^u), \quad S_u^{\text{target}} = (i_2^u) \tag{10}$$

1.3. *All action.* Based on the user sequence, predict all future actions within the time window of width $k$. In contrast to previous approaches it is more stable, as the model learns that positive action can be not the next one but happen later in time.

$$S_u^{\text{train}} = (i_1^u, i_2^u, ..., i_{T_u-k}^u), \quad S_u^{\text{target}} = (i_{T_u-k+1}^u, i_{T_u-k+2}^u ..., i_{T_u}^u) \tag{11}$$

1.4. *Dense all action.* For each subsequence predict random positive item within sliding time window of width $k$.

$$S_u^{\text{train}} = (i_1^u), \quad S_u^{\text{target}} = (i_{T_u-k+1}^u) \tag{12}$$

$$S_u^{\text{train}} = (i_1^u, i_2^u), \quad S_u^{\text{target}} = (i_{T_u-k+2}) \tag{13}$$

$$S_u^{\text{train}} = (i_1^u, i_2^u, ..., i_{T_u-k}^u), \quad S_u^{\text{target}} = (i_{T_u}^u) \tag{14}$$

2 Transformer layers. As transformer layers is the most diverse part, it was decided to test only the most basic options.

2.1. *SASRec layers.* SASRec layers use masked multi-head attention that forbids model to look in the future. What is more, post-normalization is applied, pre-normalization is used for query element of attention only. Figure describing structure can be found in Appendix A.1.

2.2. *BERT4Rec layers.* In comparison to SASRec layers, BERT4Rec layers allow model to look both in the past and in the future. Pre-normalization is applied to embeddings and in feed forward network ReLU is replaced by GeLU. Figure describing structure can be found in Appendix A.2.

## 3 Losses.

Notations:

$L = \{l_1, l_2, ..., l_N\}^T$ - vector of losses, where $l_i$ - loss for user sequence $i$

$w_i$ - weight of actual interaction $i$

$x_{n,j}$ - logit value of item j

$y_n$ - actual interaction item

$C$ - number of items in catalogue

$\mathcal{N}_n$ - sampled negatives.

### 3.1. *Softmax loss.*

$$l_n = -w_{y_n} log \frac{exp(x_{n,y_n})}{\sum_{c=1}^{C} exp(x_{n,c})} \cdot I\{y_n \neq \text{ignore index}\} \tag{15}$$

### 3.2. *Sampled softmax loss.*

$$l_n = -w_{y_n} \log \left( \frac{exp(x_{n,y_n})}{exp(x_{n,y_n}) + \sum_{j \in \mathcal{N}_n} exp(x_{n,j})} \right) I\{y_n \neq \text{ignore index}\} \tag{16}$$

### 3.3. *BCE loss.* For this loss $y_n$ is considered binary, 1 representing true item.

$$l_n = \frac{1}{|\mathcal{N}_n + 1|} \sum_{j \in \mathcal{N}_n \cup x_{n,y_n}} -w_{y_n}[y_{n,j} log\sigma(x_{n,j}) + (1 - y_{n,j}) log(1 - \sigma(x_{n,j}))] \tag{17}$$

### 3.4. *gBCE loss.* Positive logits are transformed before being used in BCE loss.

$$t - \text{calibration hyperparameter}$$

$$\alpha = \frac{1}{\#\text{ unique items in catalogue} - 1}, \quad \beta = \alpha(t(1 - \frac{1}{\alpha}) + \frac{1}{\alpha})$$

$$\text{transformed positive logits} = log(\frac{1}{\sigma^{-\beta}(\text{positive logits}) - 1}) \tag{18}$$

## 4 Negative sampling.

4.1. *Uniform sampling.* The simplest strategy of negative sampling, each item in the catalogue is given the same probability to be sampled $p = \frac{1}{N}$.

4.2. *In-batch sampling.* Each item from the batch has the same probability to be sampled $p = \frac{1}{\text{Batch size}}$.

4.3. *In-batch sampling with log-Q correction.* Same as in-batch, but logits are corrected to account for popular items being sampled more frequently.

4.4. *Mixed sampling.* Negatives are sampled uniformly and from batch in a certain proportion (ratio is a hyperparameter).

5 Similarity function.

5.1. *Dot product.* Most common way to calculate probabilities that a user will like an item is to take a scalar product of session embeddings and item embeddings.

5.2. *Cosine similarity.* Relevance is calculated based on cosine distance.

6 Item features. Features are evaluated in a separate section.

6.1. *Sum of embeddings.* Embeddings of categorical and Id features are summed up.

6.2. *DCN embedding.* Network introduced in DCN [25] and DCN-v2 [26] that applies cross layers to help model learn item interactions in a better way than simple deep network.

## 3.3 Conclusions and Results

As a result of this section, open-source modular implementation was developed that allows testing multiple hypotheses in a comfortable way. What is more, a list of component modifications that could possibly influence performance was created and integrated into framework. Next step before experiments is to outline datasets and metrics.

# 4 Performance Measurement

## 4.1 Datasets

Algorithms yield varying performance metrics on different datasets [22]. Thus, it was important to investigate several scenarios that are frequently met in articles. It was ensured that chosen datasets have a varying range of interactions and average sequence length to make study representative. Dataset statistics are shown in table 4.1, there are no cold users and items, as only user-item interactions are used for this study.

Table 4.1: Dataset summary statistics

| Dataset | Users | Items | Interactions | Avg. Sequence Length | Sparsity |
|---|---|---|---|---|---|
| ML-1M-bert | 6,040 | 3,416 | 999,611 | 165.4985 | 0.9516 |
| Beauty | 40,226 | 54,542 | 353,962 | 8.79 | 0.9998 |
| Steam | 281,428 | 13,044 | 3,488,885 | 12.398 | 0.9990 |
| ML-1M | 6040 | 3706 | 1,000,209 | 165.5975 | 0.9553 |
| ML-20M | 138,493 | 26,744 | 20,000,263 | 144.4135 | 0.9946 |
| KION | 962,179 | 15,706 | 5,476,251 | 5.6915 | 0.9996 |

First 3 datasets were taken from original BERT4Rec repository. They contain only user-items interactions, no weights and dates are provided. Hence, these datasets were not suitable for time-based validation and were used only to benchmark implementation quality. Users with at least 5 interactions were left, as it will be important for further comparisions with existing articles.

- MovieLens dataset from BERT4Rec article (ML-1M-bert) with 1 million interactions. Contains anonymous ratings of 3,416 films left by 6,040 users.
- Beauty. Filtered versions of Amazon datasets, containing ratings of cosmetics and personal care[9].
- Steam. Dataset collected from Steam online video game platform contains about 3,5 million reviews on 13,407 items.

MovieLens datasets were taken from MovieLens website [3] filtering out users with less than 2 interactions (minimum for SASRec model), following [4].

- MovieLens dataset with 1 million interactions (ML-1M). Contains anonymous ratings of 3,706 films made by 6040 users.

- MovieLens dataset with 20 million interactions (ML-20M). Contains anonymous ratings of 26,744 films made by 138,493 users.

KION dataset was collected from MTS Kion video streaming platform [16], filtering out users with less than 2 interactions, following [4].

- KION. Distinguishing feature of this dataset is that it is based on implicit user feedback rather than on explicit and that there are item features.

As it was found that many modifications are data-dependent, further study of popularity distributions for ML-1M, ML-20M and KION datasets was conducted A.3, A.4, A.1.

## 4.2 Metrics

- Accuracy metrics. It is a common practice to report accuracy metrics @10 and users usually interact with 5-20 items at a time, hence, 10 positions were taken.

  1 Recall@K - binary metric that describes which proportion of relevant items is present in top-10 recommendations.

$$\text{Recall@K} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbb{I}\left(g_u \in \text{Top@K}(f_\theta(\mathcal{S}_u))\right) \tag{19}$$

  2 NDCG@K. Compared to recall, NDCG is maximized as the most relevant not only present in the list, but also located at the top.

$$\text{NDCG@K} = \frac{1}{|U|} \sum_{u \in U} \frac{\text{DCG}_u@K}{\text{IDCG}_u@K} \tag{20}$$

$$\text{DCG}_u@K = \sum_{i=1}^{K} \frac{\text{rel}_u(i)}{\log(i+1)}, \quad \text{IDCG}_u@K = \sum_{i=1}^{K} \frac{1}{\log(i+1)}. \tag{21}$$

where, rel($i$) is 1 if item at rank $i$ is relevant, 0 - otherwise.

- Beyond-accuracy metrics.

  1 Average recommendation popularity (ARP). It measures the average popularity of items from a recommendation list.

$$\text{ARP@K} = \frac{1}{|U_t|} \sum_{u \in U_t} \frac{\sum_{i \in L_u} \phi(i)}{|L_u|} \tag{22}$$

  where, $\phi(i)$ - number of previous interactions with item $i$, $|U_t|$ - number of users in the test set, $L_u$ - top k recommended items for user $u$.

- Debiased metrics. Debiased NDCG@K and debiased Recall@K are classic metrics computed for filtered dataset. To filter dataset, most popular items with respect to their frequency are downsampled to fall into interquartile range:

$$\text{upper frequency bound} = Q_3 + 1.5 \cdot (Q_3 - Q_1) \tag{23}$$

  Such approach allows more balanced model comparison by dealing with popularity bias effects.

## 4.3 Conclusions and Results

In this part 6 datasets with diverse characteristics were selected. ML-1M-bert, Beauty and Steam datasets were used to compare model performance to existing implementations in the next section. ML-1M, ML-20M and KION to measure performance of modifications. Additionally, KION is used to identify item feature effect. To find a Pareto optimal solution, Recall@10 and ARP@10 were selected as main metrics with additional set of metrics used for control.

# 5 Experiments

## 5.1 Experiment 1: Model Quality

After implementing the model, it was important to ensure that performance is comparable to open-source options, as results are highly dependent on implementation characteristics. For that the research described in the article "A Systematic Review and Replicability Study of BERT4Rec for Sequential Recommendation"[15] was replicated. Initially it compared all existing open-source implementations of BERT4Rec on 4 datasets (Ml-1M-bert, Beauty, Steam and ML-20M). This study took the results introduced by the authors and added RecTools transformer model implementation.

### 5.1.1 Validation scheme

Leave-one-out validation scheme was taken according to the article:

- Test: last interaction for each user.
- Validation: second to last interaction for 2048 randomly selected users.
- Train: remaining interactions.

Early stopping was applied based on Recall@10 with patience 200 epochs.

### 5.1.2 Model Parameters

BERT4Rec model was tested with BERT4Rec-VAE default parameters 5.1. Adam optimizer is used, batch size 128, learning rate 0.001.

Table 5.1: Default parameters

| **Parameter** | **value** |
|---|---|
| Sequence length | 100 |
| Item masking probability | 0.15 |
| Embedding size | 256 |
| Transformer blocks | 2 |
| Attention heads | 4 |

### 5.1.3 Results

As a result, new implementation of BERT4Rec outperforms existing models at 2 datasets (MovieLens1M and Beauty), and ranks third for ML-20M and Steam (A.2-A.5). This indicates that modular implementation with untuned parameters, that was suggested in this research, is comparable to existing models and, thus, can be used for further experiments.

## 5.2 Experiment 2: Base Model

Previous experiment shows that developed modular implementation is trustworthy. Next question that arises is which configuration to take as a base one, so that it provides comparable results to SOTA models with respect to accuracy metrics and consists of simple modifications. From previous research it is evident that SASRec can produce better results than BERT4Rec. Hence, SASRec with sampled softmax loss and uniform sampling strategy was chosen. Reason for using sampled softmax instead of default BCE is performance increase, softmax was not chosen, because cross-entropy for full catalogue of items is not suitable for industrial purposes. To check whether performance metrics are comparable to existing solutions, model hyperparameters were tuned and compared to the results stated in [17] for ML-1M and from other studies applied to ML-20M dataset.

### 5.2.1 Validation scheme

Validation scheme was LOO to make comparison consistent with metrics stated in the article:

- Test: last interaction for each user.
- Validation: second to last interaction for each user.
- Train: remaining interactions.

All models were trained for maximum 200 epochs with patience 20 on validation

Recall@10. Recall was chosen following previous studies and due to the fact that aim is to maximize accuracy metrics.

### 5.2.2 Model Parameters

Parameter grid for cross-validation is presented in 5.2. Adam optimizer is used, batch size 128, learning rate 0.001.

Table 5.2: Grid search parameters

| Parameter | Value 1 | Value 2 | Value 3 |
|---|---|---|---|
| n_factors | 64 | 128 | 256 |
| session_max_len | 50 | 100 | 200 |
| n_blocks | 1 | 2 | 4 |
| n_heads | 1 | 2 | 4 |
| dropout_rate | 0.1 | 0.2 | 0.3 |
| n_negatives | 64 | 128 | 256 |

### 5.2.3 Results

Final model parameters are stated in table 5.3, metrics for final models are shown in table 5.4.

Table 5.3: Final model parameters

| Dataset | n_factors | session_max_len | n_blocks | n_heads | dropout_rate | n_negatives |
|---|---|---|---|---|---|---|
| ML-1M | 128 | 200 | 4 | 2 | 0.2 | 256 |
| ML-20M | 256 | 200 | 4 | 4 | 0.1 | 256 |
| KION | 128 | 50 | 2 | 2 | 0.1 | 256 |

Table 5.4: Final model metrics LOO

| Dataset | Recall@10 | NDCG@10 | ARP@10 | Epochs | Fit time is seconds |
|---|---|---|---|---|---|
| ML-1M | 0.3150 | 0.1770 | 0.00086 | 74 | 355 |
| ML-20M | 0.3214 | 0.1900 | 0.00069 | 105 | 15442 |
| KION | 0.3290 | 0.2060 | 0.01037 | 115 | 12687 |

Table 5.5, 5.6 show results for different models on ML-1M dataset declared in paper about gSASRec [17] and metrics on ML-20M collected from several sources. It can be seen that performance of this implementation is superior to the ones presented in previous papers for both datasets, making it a good base model.

Table 5.5: Model performance on ML-1M

| Model | R@10 | N@10 | R Diff % |
|---|---|---|---|
| **SASRec-SS** | **0.315** | **0.177** | 0.00 |
| BERT4Rec | 0.294 | 0.161 | -6.00 |
| SASRec-softmax | 0.293 | 0.169 | -6.98 |
| SASRec-BCE | 0.247 | 0.131 | -21.59 |
| SASRec-gBCE | 0.300 | 0.176 | -4.76 |
| DuoRec | 0.294 | 0.168 | -6.00 |
| ALBERT4Rec | 0.300 | 0.165 | -4.76 |
| CBiT | 0.301 | 0.169 | -4.00 |

Table 5.6: Model performance on ML-20M

| Model | R@10 | R Diff % |
|---|---|---|
| **SASRec-SS** | **0.321** | 0.00 |
| SASRec-gBCE | 0.299 | -6.85 |
| SASRec-SS-other | 0.289 | -9.97 |
| BERT4Rec | 0.234 | -27.10 |

## 5.3 Experiment 3: Benchmark of Modifications

Having defined base model, modification influence on model performance can be measured. In a best-case scenario all modified models should be tuned, however, it is infeasible in the case of limited computing power. If all models with all modifications and parameters are tuned for each dataset, there would have been 1,049,760 models. Assuming it takes 1 hour to validate each model (actually for many modifications time is much greater if ML-20M and KION datasets are computed with V100 GPU), it would take 43,740 days. That is why some simplifications were made, resulting in the following procedure:

1 For each dataset it was decided to fix model parameters that are not considered as modifications, following the article [18]. Same set of parameters was taken as defined during LOO validation (Table 5.3) to be configuration starting point.

2 Study was conducted, where modifications were changed one by one keeping everything else constant.

3 Negative-sampling strategies were investigated more closely, including change in in-batch to uniform ratio. It was done, as negative-sampling hyperpa-

rameters are known to affect results greatly.

4 Some promising combinations were evaluated based on per-component study to investigate if better pareto-optimal and accuracy results can be obtained.

Experiment was conducted in accordance to this approach for 3 datasets that have timestamps (ML-1M, ML-20M, KION). No significance tests were provided due to a large number of models tested [29].

### 5.3.1 Validation scheme

As it was stated, time-based validation is used for this study to make results useful for industrial setting. Validation scheme is the following:

- Test: last N days for each user. N is specified for each dataset individually. Split was chosen in such a way that test interactions are about 0.01% of all interactions [4]. However, for MovieLens proportion is less, as taking more days for test will degrade model performance.
- Validation: second to last N days for 2048 users.
- Train: remaining interactions.

Time split is taken with the usage of TimeRangeSplitter defined in RecTools library. Resulting number of interactions in train and test is shown in table 5.7.

Table 5.7: Time splits

| **Dataset** | **# train and val interaction** | **# test interactions** | **# n test days** |
|---|---|---|---|
| ML-1M | 993,689 | 6,383 | 110D |
| ML-20M | 19,820,002 | 54,522 | 60D |
| KION | 4,649,162 | 521,381 | 14D |

It was decided to use early-stopping by validation loss with patience 50 and maximum number of epochs - 200. Using Recall is not feasible, because both accuracy and beyond-accuracy metrics are important.

### 5.3.2 Model Parameters

Parameters defined during LOO validation (Table 5.3). For gBCE calibration hyperparameter - 0.75, window for all action and dense all action - test size.

### 5.3.3 Results

In this section only tables for ML-20M dataset were left, results for other datasets can be found in A.6-A.9. Performance was measured for models with only one component changed, including grid search over negative sampling ratios.

Table 5.8: ML-20M per-component results

| Comp | Model | R@10 | N@10 | DR@10 | DN@10 | ARP@10 | Time(s) |
|---|---|---|---|---|---|---|---|
| Baselines | Popular-7D | 0.1036 | 0.1671 | 0.0183 | 0.0184 | 0.00086 | 0 |
| | Popular | 0.0198 | 0.0478 | 0.0142 | 0.0208 | 0.00224 | 3 |
| Loss | Softmax | 0.0909 | 0.1476 | 0.0332 | 0.0392 | 0.00020 | 50170 |
| | Sampled softmax | **0.0958** | **0.1534** | **0.0392** | 0.0391 | **0.00019** | 35064 |
| | BCE | 0.0911 | 0.1469 | 0.0311 | **0.0402** | 0.00022 | 22063 |
| | gBCE | 0.0869 | 0.1370 | 0.0324 | 0.0390 | 0.00020 | 15784 |
| Sampling | Uniform | **0.0958** | **0.1534** | **0.0392** | **0.0391** | 0.00019 | 35064 |
| | InBatch | 0.0128 | 0.0152 | 0.0060 | 0.0067 | **0.00002** | 28101 |
| | InBatchLogQ | 0.0326 | 0.0500 | 0.0136 | 0.0150 | 0.00020 | 30653 |
| | Mixed-0.4 | 0.0862 | 0.1309 | 0.0308 | 0.0347 | 0.00012 | 38460 |
| | MixedLogQ-0.4 | 0.0854 | 0.1449 | 0.0296 | 0.0383 | 0.00082 | 31166 |
| Objective | Shifted | 0.0958 | 0.1534 | **0.0392** | **0.0391** | 0.00019 | 35064 |
| | MLM | 0.0717 | 0.1220 | 0.0294 | 0.0365 | 0.00020 | 13422 |
| | AllAction | **0.1137** | **0.1566** | 0.0174 | 0.0161 | **0.00015** | 79 |
| | DenseAllAction | 0.0318 | 0.0747 | 0.0146 | 0.0204 | 0.00044 | 14192 |
| Layers | SASRec | **0.0958** | **0.1534** | **0.0392** | **0.0391** | **0.00019** | 35064 |
| | BERT4Rec | 0.0717 | 0.1221 | 0.0294 | 0.0365 | 0.00020 | 13950 |
| Similarity | Dot | **0.0958** | **0.1534** | **0.0392** | **0.0391** | **0.00019** | 35064 |
| | Cosine | 0.0232 | 0.0504 | 0.0120 | 0.0162 | 0.00033 | 6998 |

Table 5.9: ML-20M negative results

| Comp | Model | R@10 | N@10 | DR@10 | DN@10 | ARP@10 | Time |
|---|---|---|---|---|---|---|---|
| Sampling | Uniform | **0.0958** | 0.1534 | **0.0392** | **0.0391** | 0.00019 | 35064 |
| | Mixed-0.2 | 0.0861 | 0.1330 | 0.0291 | 0.0350 | 0.00014 | 29562 |
| | Mixed-0.4 | 0.0862 | 0.1309 | 0.0308 | 0.0347 | 0.00012 | 38460 |
| | Mixed-0.6 | 0.0876 | 0.1301 | 0.0288 | 0.0330 | 0.00012 | 32465 |
| | Mixed-0.8 | 0.0701 | 0.1033 | 0.0280 | 0.0313 | 0.00010 | 13215 |
| | MixedLogQ-0.2 | 0.0870 | 0.1502 | 0.0295 | 0.0360 | 0.00082 | 29830 |
| | MixedLogQ-0.4 | 0.0854 | 0.1449 | 0.0296 | 0.0383 | 0.00082 | 31166 |
| | MixedLogQ-0.6 | 0.0943 | **0.1554** | 0.0306 | 0.0382 | 0.00071 | 31478 |
| | MixedLogQ-0.8 | 0.0859 | 0.1492 | 0.0307 | 0.0377 | 0.00068 | 35641 |
| | InBatch | 0.0128 | 0.0152 | 0.0060 | 0.0067 | **0.00002** | 28101 |
| | InBatchLogQ | 0.0326 | 0.0500 | 0.0136 | 0.0150 | 0.00020 | 30653 |

**Common results**

First of all, achieved results justify the need for identifying Pareto optimal solutions. In general models with higher recall have greater average recommendation popularity, requiring researchers to consider beyond-accuracy metrics. Furthermore, the need for beyond-accuracy metrics is visible from comparison with popular baselines. Especially for ML-20M dataset, popular baselines provide high accuracy metrics, but poor performance in beyond-accuracy and debiased metrics. Reason for such behavior is that ML-20M captures modern data (interactions capture period till 2015), where users frequently co-watch the same popular movies and strong trends are present. From popularity baseline investigation it can be seen that popular 7 days is a better option than popular for datasets that have enough interactions for the last 7 days, which is not the case for smaller ML-1M.

From initial analyses of the models at the Pareto frontier it can be seen that the optimal model is data-dependent. However, KION and ML-20M, that have more than 96% of items in the tail, contain shared models on their Pareto frontiers (mixed-0.6, mixed-0.8 and AllAction). For a much smaller ML-1M AllAction is the worst model of all, due to the fact that there is not enough data for it to become efficient, notably log-Q models perform well. So, as no unique best model was found, it was decided to investigate dataset characteristics and analyse individual effects of each component taking into account these differences.

To make further explanations more clear, dataset characteristics will be explained in greater details. With respect to user-item ratios datasets belong to different clusters [21]: ML-1M has balanced characteristics, in KION and ML-20M datasets number of users is much greater than number of items. What is more, frequency distributions differ significantly A.3,A.4, with ML-1M containing more items in head and mid parts.

**Per-component tendencies**

- Loss function. Classic BCE option was shown to be the worst with respect to its ability to provide balanced solutions. It makes study on negative

sampling strategy more important, as previously detailed benchmarks were based on BCE and not sampled softmax. Regarding other sampled losses switching to gBCE with 0.75 temperature leads to Pareto optimal result for KION dataset. For other datasets either sampled softmax wins or other temperature values should be considered (for ML-20M much lower temperature is required to improve performance).

- Training objective. All action training objective was shown to provide balanced results for datasets that have enough user sequences (0.2224 Recall@10 and 0.1024 ARP@10 for KION, 0.1137 Recall@10 and 0.00015 ARP@10 for ML-10M). In contrast to shifted sequence and dense all action it is not autoregressive, thus, requires more data. This result was unexpected, as PinnerFormer article claims that dense all action objective improved coverage (beyond-accuracy metric) without hurting performance. Furthermore, usage of this objective allows to reduce training time significantly without great quality drop. Next modification to consider is shifted sequence target, which shows better results than dense all action, outlining that classic SASRec has good qualities for being a baseline model.

- Transformer layers. SASRec and BERT4Rec transformer layers have similar structure, thus, performance metrics are close to each other with minor differences across datasets. However, by looking at debiased metrics it is suggested to apply SASRec layers as a first option.

- Similarity function. When choosing from dot product and cosine similarity, dot product should be taken for user to item recommendations. Reason is that cosine similarity should be applied to data from the same vector spaces, which is not the case for u2i recommendations. It is also suggested to try MLP similarity, which performed well for datasets with smaller catalogs (this similarity was implemented and can be found in repository , however, it was not possible to test it on these datasets due to computational limitations).

- Negative sampling. Results show that model performance is highly affected by sampling strategy. In general logQ correction helps to improve accuracy metrics at the same time decreasing ARP. Debiased metrics provide some further insights, indicating that logQ correction is more applicable to frequency distributions similar to ML-1M dataset. Mixed sampling strategies result in balanced performance for all datasets, as they provide both relevant and diverse data samples. However, there is no unique mixed to in-batch ratio that could be stated as optimal. A good starting option can be 0.6 for ML-20M and KION datasets and 0.4 for ML-1M.

### Modification combinations

It was further investigated if model performance could be improved by combining several modifications. Some promising combinations that were tried to achieve desired trade-off are shown in the table 5.10. Decision to try some model combination was based on results obtained in the previous part. Greater attention was dedicated to models with high accuracy metrics and Pareto optimal models. This choice was guided by the assumption that such combinations could potentially decrease normalized average recommendation popularity without hurting recall.

Table 5.10: Modification combinations

| Dataset | Model | R@10 | N@10 | DR@10 | DN@10 | ARP@10 | Time(s) |
|---|---|---|---|---|---|---|---|
| ML-1M | InBatchLogQ+gBCE | 0.0517 | 0.0816 | 0.0456 | 0.0749 | 0.00144 | 1502 |
| | MixedlogQ-0.4+gBCE | **0.0593** | 0.0821 | **0.0462** | 0.0748 | **0.00119** | 1188 |
| | Mixed-0.6+gBCE | 0.0531 | 0.0821 | 0.0491 | **0.0753** | 0.00138 | 1081 |
| | MixedlogQ-0.6+gBCE | 0.0524 | **0.0840** | 0.0436 | 0.0748 | 0.00140 | 1271 |
| | DAA+gBCE | 0.0385 | 0.0798 | 0.0316 | 0.0667 | 0.00142 | 2118 |
| ML-20M | DAA+MixedlogQ-0.6+SS | 0.0921 | 0.1489 | 0.0321 | 0.0378 | 0.00018 | 42150 |
| | AA+MixedlogQ-0.6+SS | 0.0987 | 0.1589 | 0.0356 | **0.0401** | 0.00017 | 36897 |
| | AA+Mixed-0.6+gBCE | **0.1042** | **0.1596** | **0.0362** | 0.0399 | **0.00013** | 37204 |
| | MixedlogQ-0.6+gBCE | 0.0744 | 0.1301 | 0.0287 | 0.0364 | 0.00101 | 31470 |
| | DAA+gBCE | 0.0345 | 0.0823 | 0.0168 | 0.0223 | 0.00042 | 15895 |
| KION | DAA+softmax | 0.2285 | 0.1678 | 0.0362 | 0.0218 | **0.01115** | 23142 |
| | AA+MixedlogQ-0.6+SS | 0.2256 | 0.1665 | 0.0378 | 0.0225 | 0.01142 | 26808 |
| | AA+Mixed-0.6+gBCE | **0.2298** | 0.1677 | **0.0381** | **0.0225** | 0.01128 | 27112 |
| | MixedlogQ-0.6+gBCE | 0.2209 | 0.1608 | 0.0187 | 0.0100 | 0.01531 | 31277 |
| | DAA+gBCE | 0.2295 | **0.1685** | 0.0368 | 0.0219 | 0.01120 | 24502 |

Such an approach allowed to extend set of Pareto optimal models. For ML-20

and KION datasets All Action model with mixed-0.6 negative sampling strategy and gBCE loss improved model personalization without quality decrease. For ML-1M model to be noticed is MixedlogQ-0.4 with gBCE loss.

As a result, it can be seen that by using simple modifications model quality can be improved for both accuracy and beyond-accuracy metrics. For the datasets discussed Recall@10 increased by 60%, 19%, 0.4% for ML-1M, ML-20M and KION, respectively. Furthermore, initially tuned model is not Pareto optimal for all 3 datasets. That is why it is suggested to get familiar with dataset characteristics and try several pareto-optimal models, measuring how well they perform on concrete data samples. Basing on desired degree of personalization and insights various modifications could be tested that make model discovery easier (for instance, knowledge on applicability of logQ correction or various training objectives could be helpful).

## 5.4 Experiment 4: Effect of Item Features

Even though research focuses on models applied to interactions only, it is important to state that item features are useful in increasing beyond-accuracy metrics and potentially decreasing popularity bias. Technically categorical features were added in the following way. Initially each item feature has category (for instance, genre) and several values (comedy, drama, etc.). Each value is represented as a separate embedding, for each item all embeddings corresponding to feature values it is associated with are summed up. Id embedding is, also, added to categorical embeddings. This sum is considered as final item embedding and passed to the transformer model. To show effect of item features genres and directors were chosen.

### 5.4.1 Validation Scheme

Same validation scheme as in previous experiment was applied.

### 5.4.2 Results

Figure 5.1 clearly shows that presence of features decreases average recommendation popularity for SASRec models with different losses. Furthermore, this implementation of SASRec with item features was compared to SASRec without features by MTS KION video streaming platform. A/B test results showed that features indeed help to improve model for industrial purposes. There was a positive relative lift for all platforms with p-value 0.0005.

Figure 5.1: Effect of item features on KION dataset

## 5.5 Experiment 5: Effect of DCN network Item Features

Performance change introduced by inclusion of DCN network to SASRec model with item features was not stated in previous studies, hence, it was decided to concentrate on this topic.

### 5.5.1 Validation Scheme

Same validation scheme as in experiment 3 was applied.

### 5.5.2 Model Parameters

DCN model has 5 main choice options: number of cross layers, number of deep layers, dimension of deep layers, model version (defines cross layer matrix dimensions) and model form (parallel or stacked). By checking performance results it was outlined that version 2 parallel model consistently outperforms other combinations, hence, it was taken for further research. Number of cross and deep layers was tuned, resulting into 4 deep layers of fixed dimension 256 and 3 cross layers.

### 5.5.3 Results

Introduction of more complicated categorical item embeddings improves Recall@10, however, average recommendation popularity increases. It is visible for models using only item features and item features combined with id embedding. Effect of DCN network becomes more profound with addition of greater number of categorical features.

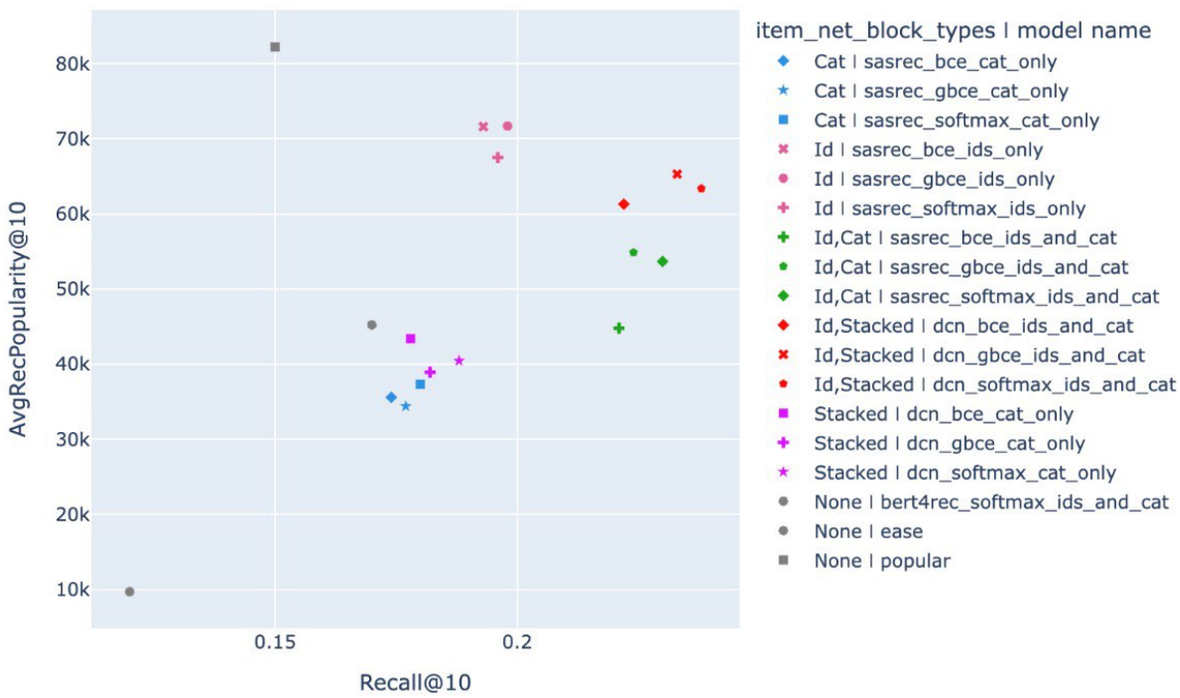


Figure 5.2: Effect of DCN item features on KION dataset

## 5.6 Conclusions and Results

It was shown that new open-source implementation is able to generate results comparable to existing frameworks with respect to accuracy metrics and better in terms of flexibility. After that, metrics were obtained that outline that baseline model can reach high performance on leave-one-out validation used by the most of articles. Experiment 3 described benchmarking procedure and offered some insights into how to define optimal model. Deep investigation of accuracy and beyond-accuracy metrics on open-source datasets for the first time was conducted for training objectives and negative sampling strategies with sampled softmax. In experiment 4 missing part on item features was addressed, as their inclusion is of utmost importance for personalized recommendations. Their ability to shift Pareto boundary was shown and supported by A/B test. Topic of item features was further expanded by introduction of DCN item features. As a result, 4960 lines of python code were written (195 KB) and 2897 lines in jupyter notebooks (1.69 MB).

# 6 Conclusion

As a result, state-of-the-art implementation of the baseline models (SASRec and BERT4Rec) was developed that was released in an open-source framework. Implementation involves multiple modular configurations that were presented in recent studies and that cannot be found in existing frameworks. Due to its structure, framework allows the testing of a large number of model modifications without rewriting base code, thus, it is useful for extended research in this topic. Model quality was proven by several experiments with fixed parameters and using LOO validation.

This study introduced comparison of transformer model architectures that were not benchmarked against each other in previous works. Insights were obtained, allowing to outline best model combinations for industrial settings that should be tried at the first place. Additionally it was shown by offline evaluation and A/B test that adding item features improves industrial model quality.

Even though in this study the most baseline model components were outlined, available computational resources did not allow testing all popular model solutions. For future research, first off, it is proposed to try more transformer model modifications. For instance, layers used in SOTA architectures (FuXI, HSTU), implement contrastive losses, measure the effect of other positional encodings (sinusoidal, relative). Another suggestion is to increase robustness of the study by making more model runs. What is more, performance of Pareto optimal models could be benchmarked against SOTA models to investigate what quality increase could be obtained through the usage of this modular approach.

# A Appendix

## Transformer layers

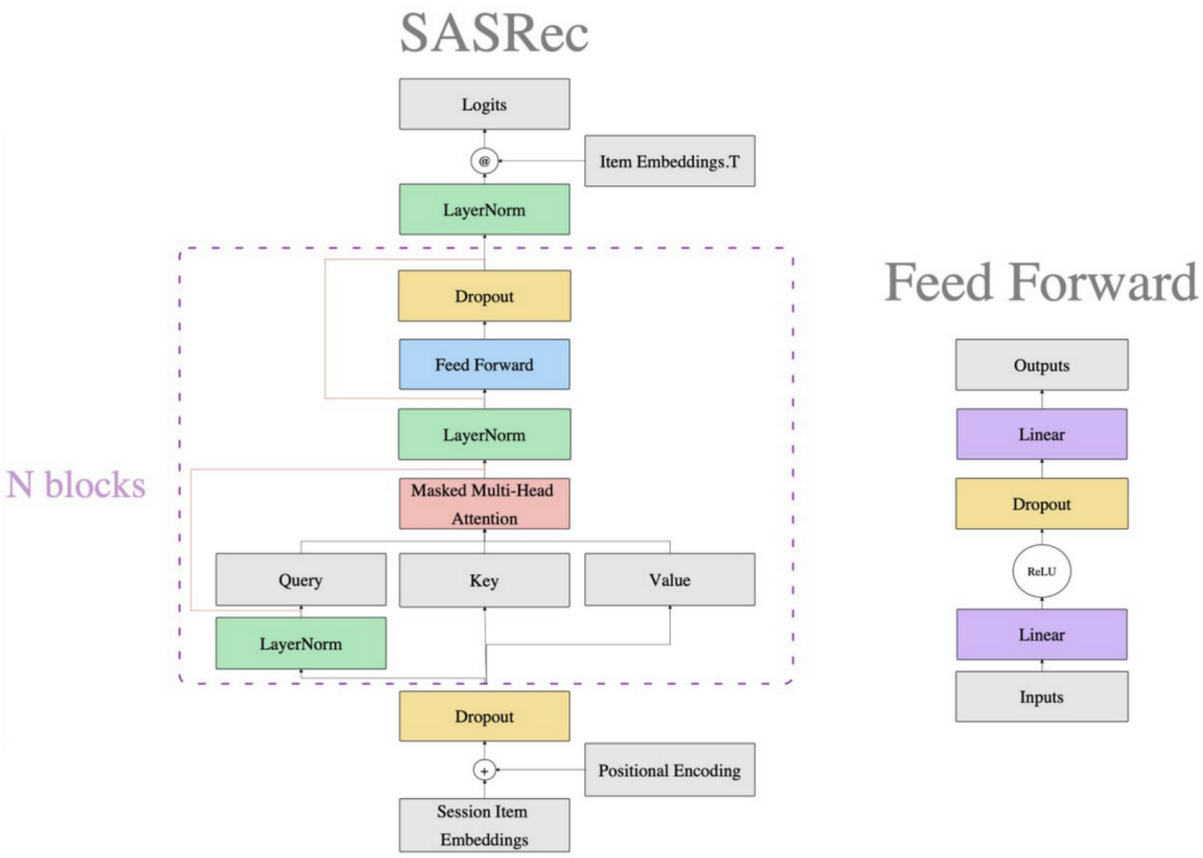


Figure A.1: SASRec transformer layers

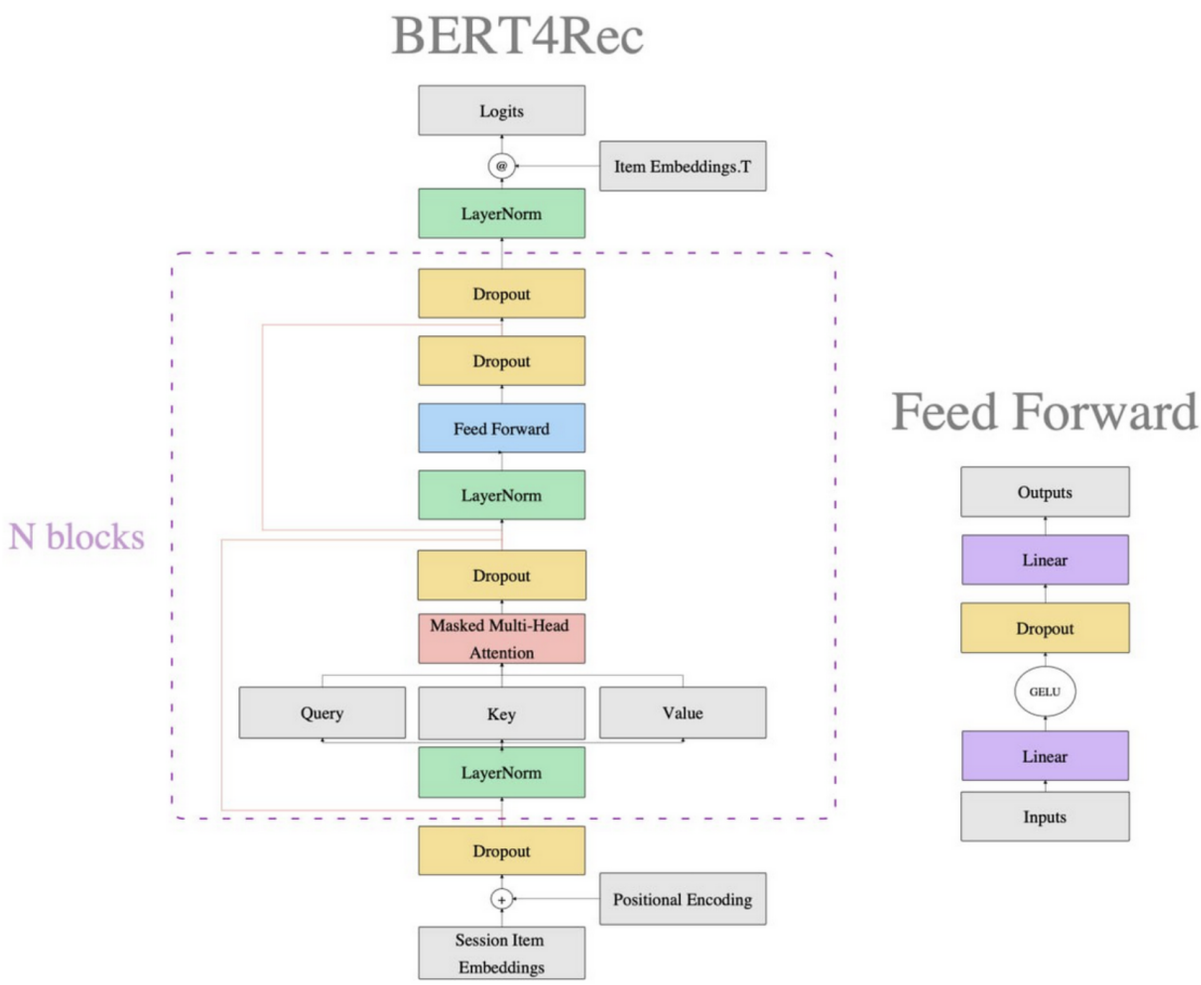


Figure A.2: BERT4Rec transformer layers

## Dataset distributions

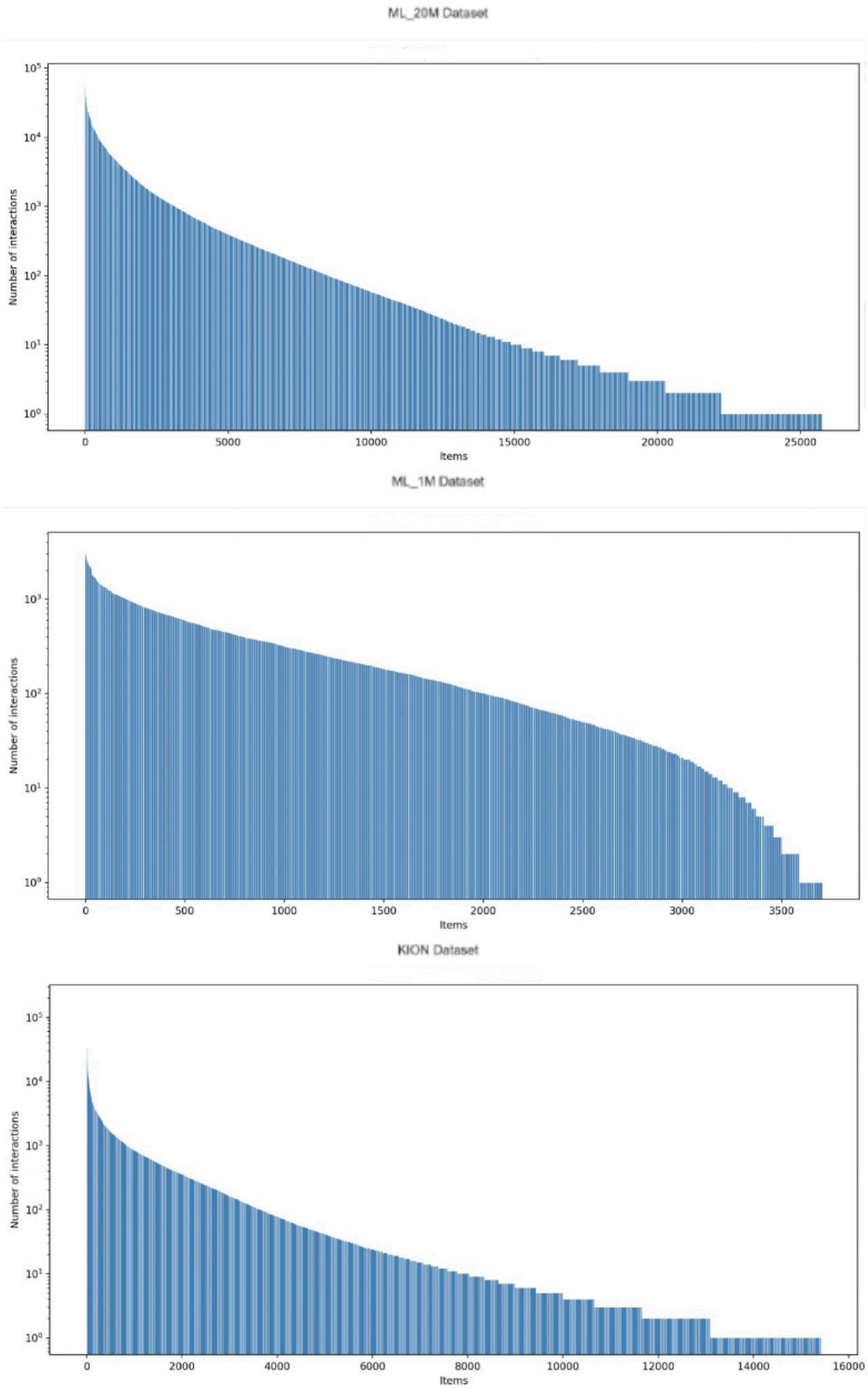


Figure A.3: Dataset frequency distributions

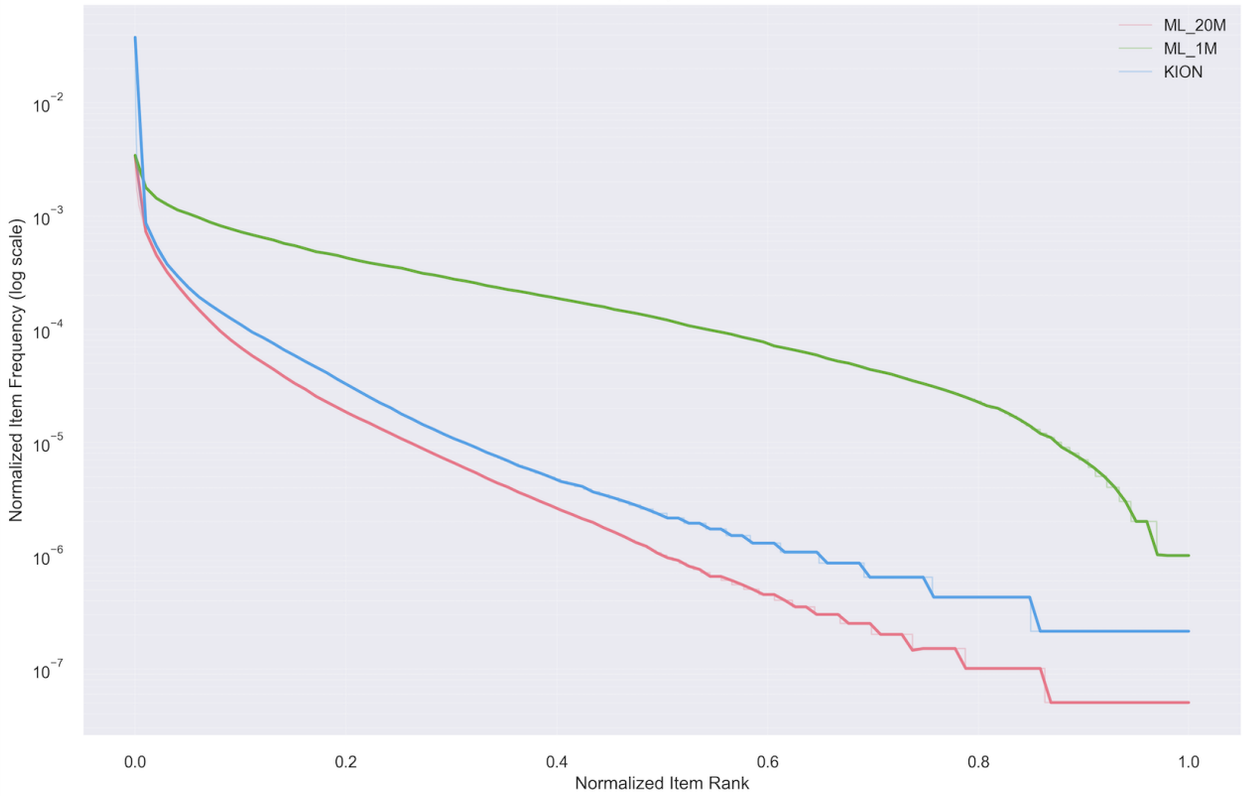


Figure A.4: Item popularity distribution

Table A.1: Distribution cohorts

| **Dataset** | **Cohort** | **Items (%)** |
|---|---|---|
| ML_20M | Head | 0.21 |
| | Mid | 0.65 |
| | Tail | 99.14 |
| ML_1M | Head | 5.95 |
| | Mid | 13.54 |
| | Tail | 80.41 |
| KION | Head | 0.36 |
| | Mid | 3.23 |
| | Tail | 96.41 |

## Experiment 1

Table A.2: ML-20M Dataset

| Model | Recall@10(PS) | NDCG@10(PS) | Recall@10 | NDCG@10 | Training Time(s) |
|---|---|---|---|---|---|
| BERT4Rec original | 0.4027 | 0.2193 | 0.0939 | 0.0474 | 6029 |
| BERT4Rec RecBole | 0.4611 | 0.2589 | 0.0906 | 0.0753 | 519666 |
| BERT4Rec-VAE | **0.7409** | **0.5259** | **0.2886** | **0.1732** | 23030 |
| ber4rec_repro | 0.7127 | 0.4805 | 0.2393 | 0.1310 | 44610 |
| ber4rec_repro(long) | 0.7268 | 0.4980 | 0.2514 | 0.1456 | 39632 |
| BERT4Rec RecTools | 0.7071 | 0.4860 | 0.2400 | 0.1396 | 29403 |

Table A.3: Beauty Dataset

| Model | Recall@10(PS) | NDCG@10(PS) | Recall@10 | NDCG@10 | Training Time(s) |
|---|---|---|---|---|---|
| BERT4Rec original | 0.1099 | 0.0567 | 0.0163 | 0.0079 | 3249 |
| BERT4Rec RecBole | 0.1996 | 0.1103 | 0.0158 | 0.0079 | 11024 |
| BERT4Rec-VAE | 0.2339 | 0.1407 | 0.0331 | 0.0188 | 21426 |
| ber4rec_repro | 0.1891 | 0.0919 | 0.0166 | 0.0080 | 14497 |
| BERT4Rec RecTools | **0.2506** | **0.1555** | **0.0434** | **0.0242** | 15786 |

Table A.4: Steam Dataset

| Model | Recall@10(PS) | NDCG@10(PS) | Recall@10 | NDCG@10 | Training Time(s) |
|---|---|---|---|---|---|
| BERT4Rec original | 0.2148 | 0.1064 | 0.0737 | 0.0375 | 4847 |
| BERT4Rec RecBole | 0.2325 | 0.1177 | 0.0744 | 0.0377 | 83816 |
| BERT4Rec-VAE | 0.3520 | 0.1941 | 0.1237 | 0.0526 | 65303 |
| ber4rec_repro | **0.3978** | **0.2219** | **0.1361** | **0.0734** | 117651 |
| BERT4Rec RecTools | 0.2856 | 0.1485 | 0.1010 | 0.0518 | 53293 |

Table A.5: ML-1M Dataset

| Model | Recall@10(PS) | NDCG@10(PS) | Recall@10 | NDCG@10 | Training Time(s) |
|---|---|---|---|---|---|
| BERT4Rec original | 0.5215 | 0.3042 | 0.1518 | 0.0806 | 2665 |
| BERT4Rec RecBole | 0.4562 | 0.2589 | 0.1061 | 0.0546 | 20499 |
| BERT4Rec-VAE | 0.6698 | 0.4533 | 0.2394 | 0.1314 | 1085 |
| ber4rec_repro | 0.6865 | 0.4602 | 0.2584 | 0.1392 | 3679 |
| ber4rec_repro(long) | 0.6975 | 0.4751 | 0.2821 | 0.1516 | 2889 |
| BERT4Rec RecTools | **0.7151** | **0.5020** | **0.2921** | **0.1618** | 602 |

## Experiment 3

Table A.6: ML-1M per-component results

| Comp | Model | R@10 | N@10 | DR@10 | DN@10 | ARP@10 | Time(s) |
|---|---|---|---|---|---|---|---|
| Baselines | Popular-7D | 0.0137 | 0.0315 | 0.0138 | 0.0304 | 0.00068 | 0 |
| | Popular | 0.0421 | 0.0791 | 0.0393 | 0.0705 | 0.00216 | 0 |
| Loss | Softmax | 0.0425 | 0.0646 | 0.0330 | 0.0536 | **0.00067** | 279 |
| | Sampled softmax | 0.0415 | 0.0640 | 0.0338 | 0.0553 | 0.00067 | 509 |
| | BCE | 0.0328 | 0.0591 | 0.0266 | 0.0545 | 0.00077 | 475 |
| | gBCE-0.75 | **0.0432** | **0.0657** | **0.0382** | **0.0589** | 0.00075 | 687 |
| Sampling | Uniform | 0.0415 | 0.0640 | 0.0338 | 0.0553 | 0.00067 | 509 |
| | InBatch | 0.0361 | 0.0576 | 0.0318 | 0.0519 | **0.00055** | 1103 |
| | InBatchLogQ | 0.0568 | 0.0847 | 0.0462 | 0.0752 | 0.00122 | 1703 |
| | Mixed-0.4 | 0.0322 | 0.0578 | 0.0305 | 0.0514 | 0.00061 | 989 |
| | MixedLogQ-0.4 | **0.0668** | **0.0929** | **0.0558** | **0.0802** | 0.00131 | 1671 |
| Objective | Shifted | **0.0415** | 0.0640 | 0.0338 | 0.0553 | 0.00067 | 509 |
| | MLM | 0.0387 | **0.0693** | **0.0368** | **0.0650** | 0.00082 | 1112 |
| | AllAction | 0.0045 | 0.0120 | 0.0029 | 0.0107 | **0.00043** | 6 |
| | DenseAllAction | 0.0353 | 0.0607 | 0.0312 | 0.0542 | 0.00068 | 2127 |
| Layers | SASRec | **0.0415** | **0.0640** | **0.0338** | **0.0553** | 0.00067 | 509 |
| | BERT4Rec | 0.0348 | 0.0547 | 0.0296 | 0.0480 | **0.00061** | 532 |
| Similarity | Dot | **0.0415** | **0.0640** | **0.0338** | **0.0553** | **0.00067** | 509 |
| | Cosine | 0.0358 | 0.0626 | 0.0324 | 0.0532 | 0.00124 | 383 |

Table A.7: KION per-component results

| Comp | Model | R@10 | N@10 | DR@10 | DN@10 | ARP@10 | Time(s) |
|---|---|---|---|---|---|---|---|
| Baselines | Popular-7D | 0.2089 | 0.1429 | 0.0095 | 0.0049 | 0.01400 | 0 |
| | Popular | 0.1784 | 0.1284 | 0.0097 | 0.0050 | 0.01663 | 1 |
| Loss | Softmax | 0.2292 | 0.1688 | **0.0390** | **0.0239** | **0.01089** | 18980 |
| | Sampled softmax | 0.2289 | 0.1683 | 0.0387 | 0.0230 | 0.01136 | 25402 |
| | BCE | 0.2299 | 0.1691 | 0.0356 | 0.0211 | 0.01204 | 22880 |
| | gBCE | **0.2313** | **0.1710** | 0.0356 | 0.0211 | 0.01201 | 18179 |
| Sampling | Uniform | **0.2289** | **0.1683** | 0.0387 | 0.0230 | 0.01136 | 25402 |
| | InBatch | 0.1716 | 0.1212 | 0.0404 | 0.0255 | **0.00603** | 35829 |
| | InBatchLogQ | 0.1484 | 0.0936 | 0.0083 | 0.0045 | 0.00854 | 44162 |
| | Mixed-0.4 | 0.2101 | 0.1540 | **0.0503** | **0.0312** | 0.00836 | 46328 |
| | MixedLogQ-0.4 | 0.2188 | 0.1592 | 0.0189 | 0.0097 | 0.01549 | 65448 |
| Objective | Shifted | **0.2289** | **0.1683** | **0.0387** | **0.0230** | 0.01136 | 25402 |
| | MLM | 0.2184 | 0.1594 | 0.0284 | 0.0160 | 0.01129 | 10662 |
| | AllAction | 0.2224 | 0.1578 | 0.0250 | 0.0142 | **0.01024** | 766 |
| | DenseAllAction | 0.2267 | 0.1664 | 0.0354 | 0.0205 | 0.01126 | 24417 |
| Layers | SASRec | **0.2289** | **0.1683** | **0.0387** | **0.0230** | 0.01136 | 25402 |
| | BERT4Rec | 0.2254 | 0.1660 | 0.0338 | 0.0198 | **0.01125** | 14284 |
| Similarity | Dot | **0.2289** | **0.1683** | **0.0387** | **0.0230** | **0.01136** | **25402** |
| | Cosine | 0.1911 | 0.1367 | 0.0133 | 0.0074 | 0.01388 | 5566 |

Table A.8: ML-1M negative results

| Comp | Model | R@10 | N@10 | DR@10 | DN@10 | ARP@10 | Time |
|---|---|---|---|---|---|---|---|
| Sampling | Uniform | 0.0415 | 0.0640 | 0.0338 | 0.0553 | 0.00067 | 509 |
| | Mixed-0.2 | 0.0449 | 0.0702 | 0.0393 | 0.0596 | 0.00073 | 1450 |
| | Mixed-0.4 | 0.0320 | 0.0578 | 0.0305 | 0.0514 | 0.00061 | 989 |
| | Mixed-0.6 | 0.0309 | 0.0604 | 0.0262 | 0.0535 | 0.00063 | 1487 |
| | Mixed-0.8 | 0.0303 | 0.0563 | 0.0278 | 0.0520 | 0.00062 | 1647 |
| | MixedLogQ-0.2 | 0.0589 | 0.0923 | 0.0473 | 0.0799 | 0.00128 | 1570 |
| | MixedLogQ-0.4 | **0.0668** | **0.0929** | **0.0558** | **0.0802** | 0.00131 | 1671 |
| | MixedLogQ-0.6 | 0.0516 | 0.0870 | 0.0477 | 0.0792 | 0.00120 | 1586 |
| | MixedLogQ-0.8 | 0.0542 | 0.0814 | 0.0435 | 0.0697 | 0.00126 | 1539 |
| | InBatch | 0.0361 | 0.0576 | 0.0318 | 0.0519 | **0.00055** | 1103 |
| | InBatchLogQ | 0.0568 | 0.0847 | 0.0462 | 0.0752 | 0.00122 | 1703 |

Table A.9: KION negative results

| Comp | Model | R@10 | N@10 | DR@10 | DN@10 | ARP@10 | Time |
|---|---|---|---|---|---|---|---|
| Sampling | Uniform | **0.2289** | **0.1683** | 0.0387 | 0.0230 | 0.01136 | 25402 |
| | Mixed-0.2 | 0.2156 | 0.1581 | 0.0416 | 0.0301 | 0.00921 | 51892 |
| | Mixed-0.4 | 0.2101 | 0.1540 | 0.0503 | 0.0312 | 0.00836 | 46328 |
| | Mixed-0.6 | 0.2081 | 0.1463 | **0.0544** | **0.0337** | 0.00782 | 48332 |
| | Mixed-0.8 | 0.1984 | 0.1357 | 0.0498 | 0.0306 | 0.00653 | 50345 |
| | MixedLogQ-0.2 | 0.2205 | 0.1590 | 0.0178 | 0.0088 | 0.01535 | 65808 |
| | MixedLogQ-0.4 | 0.2188 | 0.1592 | 0.0189 | 0.0097 | 0.01549 | 65448 |
| | MixedLogQ-0.6 | 0.2204 | 0.1610 | 0.0196 | 0.0101 | 0.01529 | 70553 |
| | MixedLogQ-0.8 | 0.2233 | 0.1652 | 0.0195 | 0.0100 | 0.01529 | 67038 |
| | InBatch | 0.1716 | 0.1212 | 0.0404 | 0.0255 | **0.00603** | 35829 |
| | InBatchLogQ | 0.1484 | 0.0936 | 0.0083 | 0.0045 | 0.00854 | 44162 |

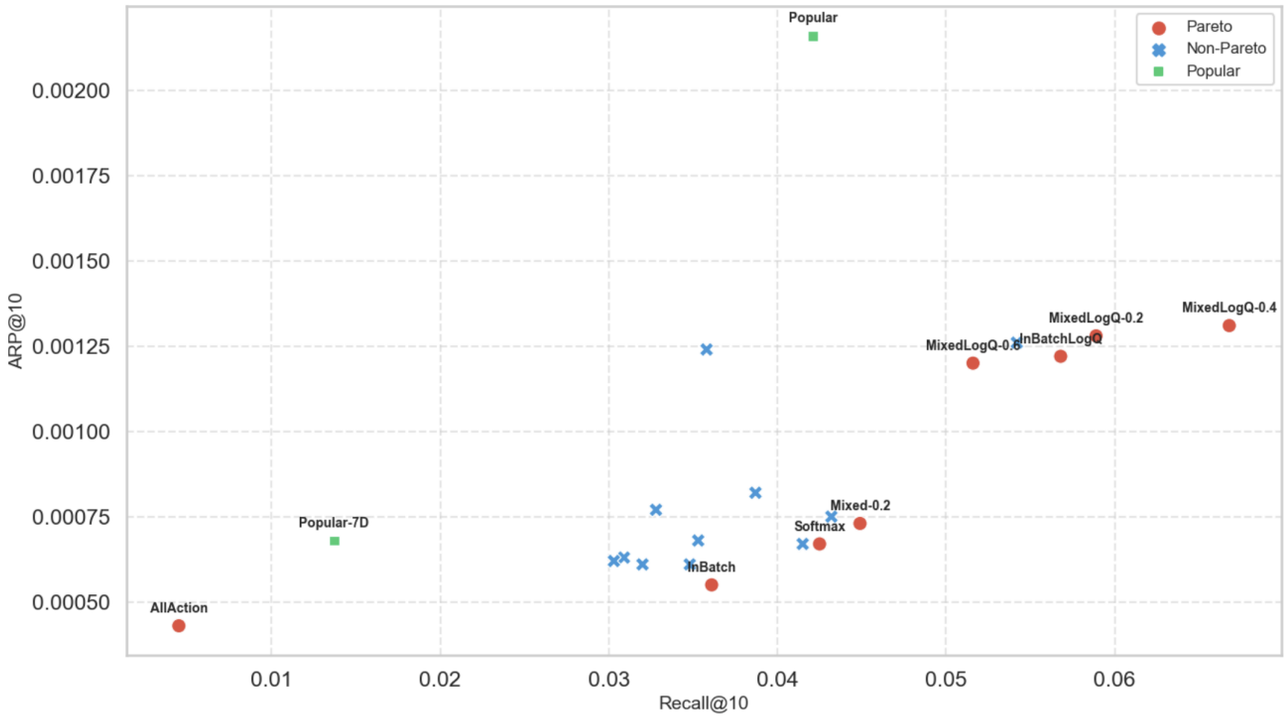


Figure A.5: Modification results ML-1M

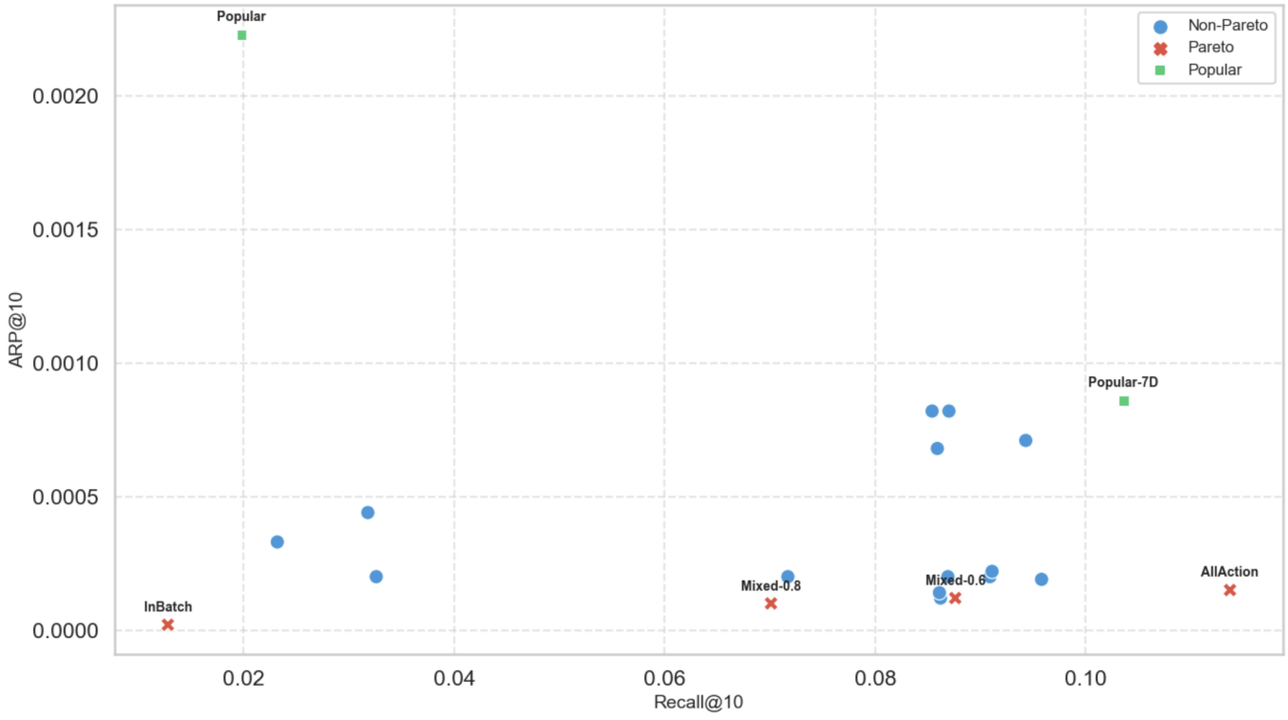


Figure A.6: Modification results ML-20M

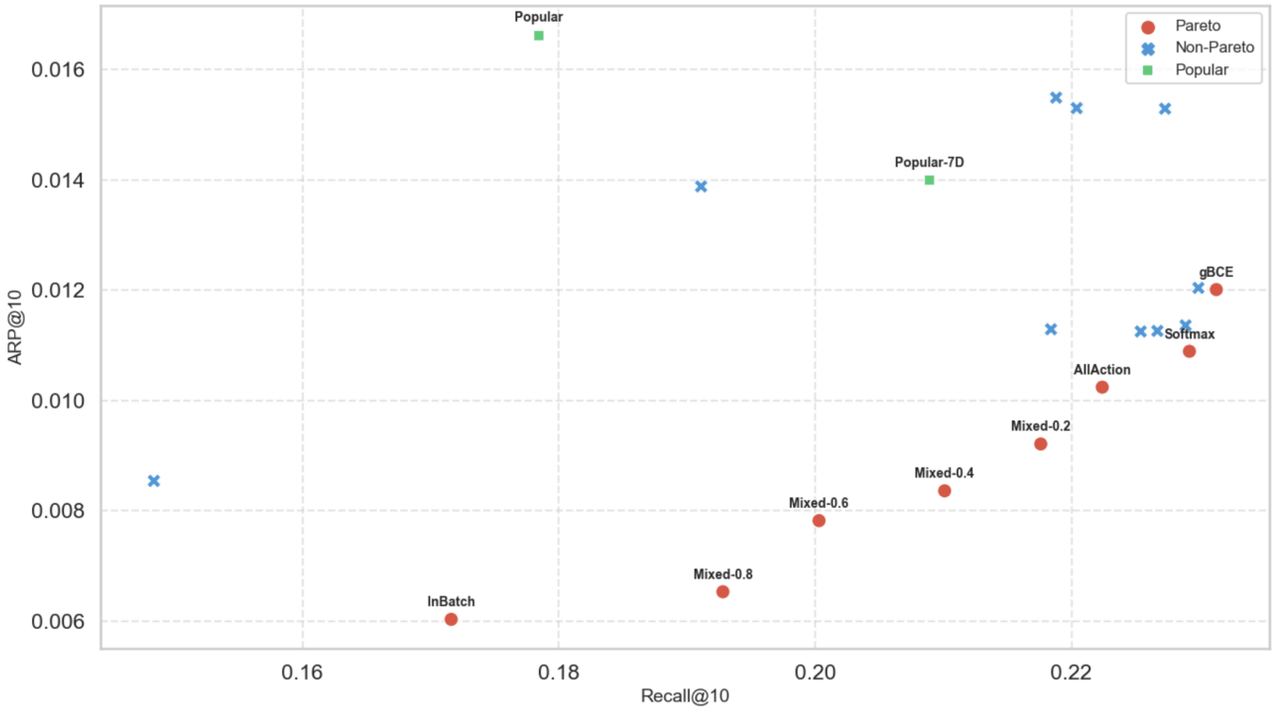


Figure A.7: Modification results KION

# References


[1] Rocío Cañamares and Pablo Castells. "Should I Follow the Crowd? A Probabilistic Analysis of the Effectiveness of Popularity in Recommender Systems". In: *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. SIGIR '18. Ann Arbor, MI, USA: Association for Computing Machinery, 2018, pp. 415–424. ISBN: 9781450356572. DOI: 10.1145/3209978.3210014. URL: https://doi.org/10.1145/3209978.3210014.

[2] Shivangi Gheewala, Shuxiang Xu, Soonja Yeom, and Sumbal Maqsood. "Exploiting deep transformer models in textual review based recommender systems". In: *Expert Syst. Appl.* 235.C (Jan. 2024). ISSN: 0957-4174. DOI: 10.1016/j.eswa.2023.121120. URL: https://doi.org/10.1016/j.eswa.2023.121120.

[3] F. Maxwell Harper and Joseph A. Konstan. "The MovieLens Datasets: History and Context". In: *ACM Trans. Interact. Intell. Syst.* 5.4 (Dec. 2015). ISSN: 2160-6455. DOI: 10.1145/2827872. URL: https://doi.org/10.1145/2827872.

[4] Balázs Hidasi and Ádám Tibor Czapp. "Widespread Flaws in Offline Evaluation of Recommender Systems". In: *Proceedings of the 17th ACM Conference on Recommender Systems*. RecSys '23. ACM, Sept. 2023, pp. 848–855. DOI: 10.1145/3604915.3608839. URL: http://dx.doi.org/10.1145/3604915.3608839.

[5] Yitong Ji, Aixin Sun, Jie Zhang, and Chenliang Li. "A Critical Study on Data Leakage in Recommender System Offline Evaluation". In: *ACM Transactions on Information Systems* 41.3 (Feb. 2023), pp. 1–27. ISSN: 1558-2868. DOI: 10.1145/3569930. URL: http://dx.doi.org/10.1145/3569930.

[6] Wang-Cheng Kang and Julian J. McAuley. "Self-Attentive Sequential Recommendation". In: *CoRR* abs/1808.09781 (2018). arXiv: 1808.09781. URL: http://arxiv.org/abs/1808.09781.

[7] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. 2020. arXiv: 1909.11942 [cs.CL]. URL: https://arxiv.org/abs/1909.11942.

[8] Haokai Ma, Ruobing Xie, Lei Meng, Fuli Feng, Xiaoyu Du, Xingwu Sun, Zhanhui Kang, and Xiangxu Meng. *Negative Sampling in Recommendation: A Survey and Future Directions*. 2024. arXiv: 2409.07237 [cs.IR]. URL: https://arxiv.org/abs/2409.07237.

[9] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. *Image-based Recommendations on Styles and Substitutes*. 2015. arXiv: 1506.04757 [cs.CV]. URL: https://arxiv.org/abs/1506.04757.

[10] Nanna Bach Munkholm, Robert Alphinas, and Torben Tambo. *Collaborative filtering, K-nearest neighbor and cosine similarity in home decor recommender systems*. 2024. arXiv: 2402.06233 [cs.IR]. URL: https://arxiv.org/abs/2402.06233.

[11] Naoto Ohsaka and Riku Togashi. "Curse of "Low" Dimensionality in Recommender Systems". In: *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '23. ACM, July 2023, pp. 537–547. DOI: 10.1145/3539618.3591659. URL: http://dx.doi.org/10.1145/3539618.3591659.

[12] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. *PinnerFormer: Sequence Modeling for User Representation at Pinterest*. 2022. arXiv: 2205.04507 [cs.LG]. URL: https://arxiv.org/abs/2205.04507.

[13] Bansari Patel, Palak Desai, and Urvi Panchal. "Methods of recommender system: A review". In: *2017 International Conference on Innovations in In-*

*formation, Embedded and Communication Systems (ICIIECS)*. 2017, pp. 1–4. DOI: 10.1109/ICIIECS.2017.8275856.

[14] Roberto Pellegrini, Wenjie Zhao, and Iain Murray. "Don't recommend the obvious: estimate probability ratios". In: *Proceedings of the 16th ACM Conference on Recommender Systems*. RecSys '22. Seattle, WA, USA: Association for Computing Machinery, 2022, pp. 188–197. ISBN: 9781450392785. DOI: 10.1145/3523227.3546753. URL: https://doi.org/10.1145/3523227.3546753.

[15] Aleksandr Petrov and Craig Macdonald. *A Systematic Review and Replicability Study of BERT4Rec for Sequential Recommendation*. 2022. arXiv: 2207.07483 [cs.IR]. URL: https://arxiv.org/abs/2207.07483.

[16] Aleksandr Petrov, Ildar Safilo, Daria Tikhonovich, and Dmitry Ignatov. *MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation*. 2022. arXiv: 2209.00325 [cs.IR]. URL: https://arxiv.org/abs/2209.00325.

[17] Aleksandr Vladimirovich Petrov and Craig Macdonald. "gSASRec: Reducing Overconfidence in Sequential Recommendation Trained with Negative Sampling". In: *Proceedings of the 17th ACM Conference on Recommender Systems*. RecSys '23. ACM, Sept. 2023, pp. 116–128. DOI: 10.1145/3604915.3608783. URL: http://dx.doi.org/10.1145/3604915.3608783.

[18] Arushi Prakash, Dimitrios Bermperidis, and Srivas Chennu. *Evaluating Performance and Bias of Negative Sampling in Large-Scale Sequential Recommendation Models*. 2024. arXiv: 2410.17276 [cs.IR].

[19] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. *Recommender Systems with Generative Retrieval*. 2023. arXiv: 2305.05065 [cs.IR]. URL: https://arxiv.org/abs/2305.05065.

[20] Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. *Neural Collaborative Filtering vs. Matrix Factorization Revisited*. 2020. arXiv: 2005.09683 [cs.IR]. URL: https://arxiv.org/abs/2005.09683.

[21] Valeriy Shevchenko, Nikita Belousov, Alexey Vasilev, Vladimir Zholobov, Artyom Sosedka, Natalia Semenova, Anna Volodkevich, Andrey Savchenko, and Alexey Zaytsev. "From Variability to Stability: Advancing RecSys Benchmarking Practices". In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. KDD '24. ACM, Aug. 2024, pp. 5701–5712. DOI: 10.1145/3637528.3671655. URL: http://dx.doi.org/10.1145/3637528.3671655.

[22] Valeriy Shevchenko, Nikita Belousov, Alexey Vasilev, Vladimir Zholobov, Artyom Sosedka, Natalia Semenova, Anna Volodkevich, Andrey Savchenko, and Alexey Zaytsev. "From Variability to Stability: Advancing RecSys Benchmarking Practices". In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. KDD '24. ACM, Aug. 2024, pp. 5701–5712. DOI: 10.1145/3637528.3671655. URL: http://dx.doi.org/10.1145/3637528.3671655.

[23] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. *BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer*. 2019. arXiv: 1904.06690 [cs.IR]. URL: https://arxiv.org/abs/1904.06690.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[25] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. *Deep Cross Network for Ad Click Predictions*. 2017. arXiv: 1708.05123 [cs.LG]. URL: https://arxiv.org/abs/1708.05123.

[26] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. "DCN V2: Improved Deep amp; Cross Network and Practical Lessons for Web-scale Learning to Rank Systems". In: *Proceedings of the Web Conference 2021*. WWW '21. ACM, Apr. 2021. DOI: 10.1145/3442381.3450078. URL: http://dx.doi.org/10.1145/3442381.3450078.

[27] Ji Yang, Xinyang Yi, Derek Zhiyuan Cheng, Lichan Hong, Yang Li, Simon Xiaoming Wang, Taibai Xu, and Ed H. Chi. "Mixed Negative Sampling for Learning Two-tower Neural Networks in Recommendations". In: *Companion Proceedings of the Web Conference 2020*. WWW '20. Taipei, Taiwan: Association for Computing Machinery, 2020, pp. 441–447. ISBN: 9781450370240. DOI: 10.1145/3366424.3386195. URL: https://doi.org/10.1145/3366424.3386195.

[28] Yufei Ye, Wei Guo, Jin Yao Chin, Hao Wang, Hong Zhu, Xi Lin, Yuyang Ye, Yong Liu, Ruiming Tang, Defu Lian, and Enhong Chen. *FuXi-$\alpha$: Scaling Recommendation Model with Feature Interaction Enhanced Transformer*. 2025. arXiv: 2502.03036 [cs.IR]. URL: https://arxiv.org/abs/2502.03036.

[29] Jiaqi Zhai, Zhaojie Gong, Yueming Wang, Xiao Sun, Zheng Yan, Fu Li, and Xing Liu. "Revisiting Neural Retrieval on Accelerators". In: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. KDD '23. ACM, Aug. 2023, pp. 5520–5531. DOI: 10.1145/3580305.3599897. URL: http://dx.doi.org/10.1145/3580305.3599897.

[30] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. *Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations*. 2024. arXiv: 2402.17152 [cs.LG]. URL: https://arxiv.org/abs/2402.17152.

[31] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. "S3-Rec: Self-Supervised Learning for Sequential Recommendation with Mutual Information Maximization". In: *Proceedings of the 29th ACM International Conference on Information amp; Knowledge Management*. CIKM '20. ACM, Oct. 2020, pp. 1893–1902. DOI: 10.1145/3340531.3411954. URL: http://dx.doi.org/10.1145/3340531.3411954.

[32] Jieming Zhu, Quanyu Dai, Liangcai Su, Rong Ma, Jinyang Liu, Guohao Cai, Xi Xiao, and Rui Zhang. *BARS: Towards Open Benchmarking for Recommender Systems*. 2022. arXiv: 2205.09626 [cs.IR]. URL: https://arxiv.org/abs/2205.09626.